

Pre-entrega Módulo 2

Arquitectura Multiagente Manager–Worker · Proyecto Renovadas

Ingrid Ledesma

Configuración e Interfaces Agénticas

Este documento acredita la implementación de una arquitectura Manager–Worker en n8n para el proyecto integrador de la carrera de AI Automation de CoderHouse. Un Workflow Manager central clasifica la intención del cliente y delega la resolución a uno de cuatro Workers especializados, implementados como sub-workflows independientes invocados vía Execute Workflow con espera de finalización activada.

Componente	Detalle
Workflow Manager	Manager_Renovadas — Chat Trigger + AI Agent clasificador + IF + Switch + 4x Execute Workflow
Workers (sub-workflows)	Worker_Catalogo, Worker_Tracking, Worker_Pedidos, Worker_Escalamiento (4, mínimo pedido: 2)
Trigger de cada Worker	Execute Workflow Trigger
Wait for child to finish	Activado ("Wait For Sub-Workflow Completion") en los 4 nodos Execute Workflow
Log de trazabilidad	Slack — nodo 'Send a message' / 'Human in the loop' según el Worker

1. Lienzo del Workflow Manager

El Manager recibe el mensaje del cliente por Chat Trigger, lo clasifica con un AI Agent liviano (Router), parsea la clasificación con un nodo Code, decide con un IF si el riesgo es alto (deriva directo a Escalamiento) y, si no, usa un Switch para enrutar al Worker correspondiente según la intención detectada. Las 4 llamadas Execute Workflow ("Call 'Worker_...'") son visibles y convergen en un nodo Edit Fields que homogeneiza la respuesta final.



Figura 1 — Lienzo completo del Manager_Renovadas, con el nodo Execute Workflow visible en sus 4 ramas.

2. Configuración del nodo Execute Workflow (Handoff de datos)

Cada nodo "Call 'Worker_...'" pasa un contrato de interfaz JSON limpio de 4 campos (original_input, intent, confidence, risk) hacia el Worker seleccionado. La opción "Wait For Sub-Workflow Completion" está activada (toggle verde, visible en la captura), lo que pausa la ejecución del Manager hasta que el Worker devuelva su resultado estructurado.

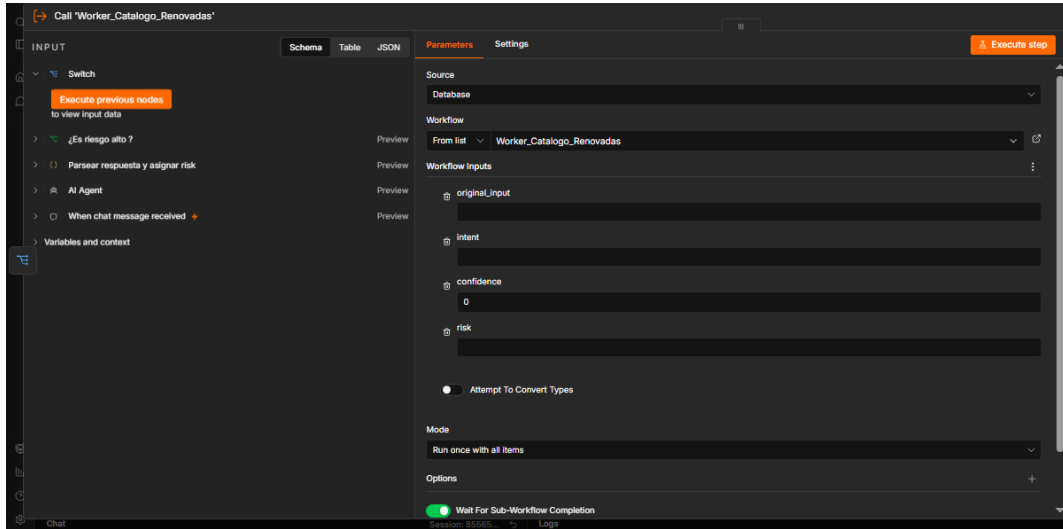


Figura 2 — Nodo Execute Workflow: parámetros de entrada (Workflow Inputs) y "Wait For Sub-Workflow Completion" activado.

3. Workers (sub-workflows especializados)

Se implementaron 4 Workers independientes, cada uno en su propio lienzo, iniciando obligatoriamente con un nodo Execute Workflow Trigger ("Entrada desde Manager") y finalizando con un nodo de salida que devuelve un JSON estructurado.

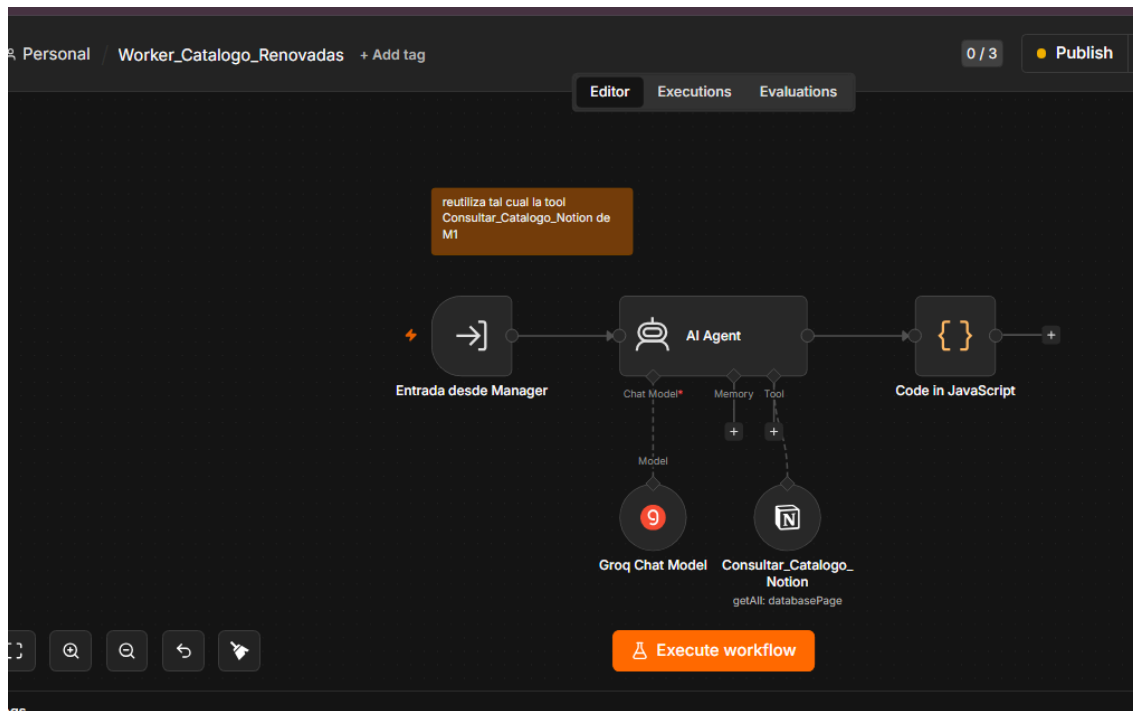


Figura 3 — Worker_Catalogo_Renovadas: Execute Workflow Trigger → AI Agent (con tool Consultar_Catalogo_Notion) → Code in JavaScript (salida).

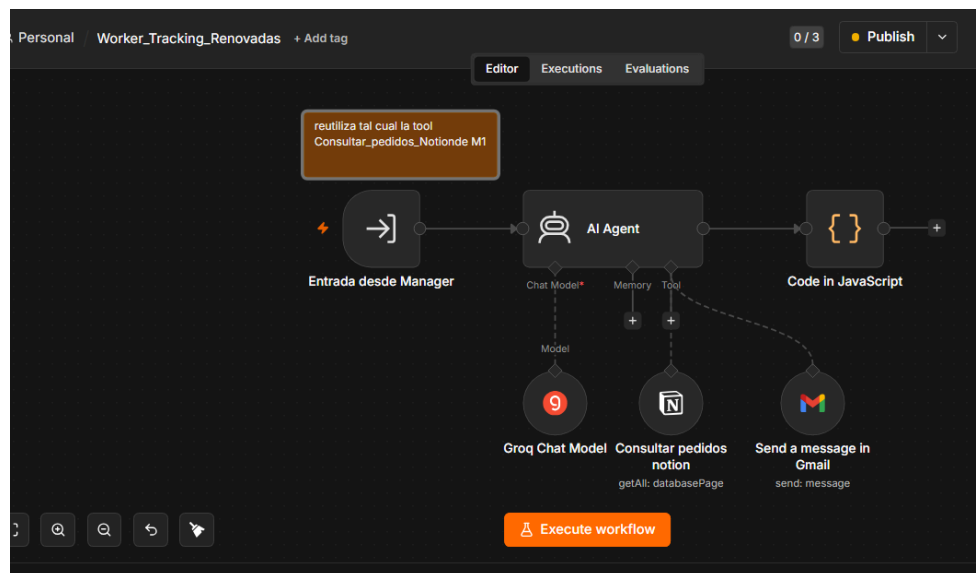


Figura 4 — Worker_Tracking_Renovadas: Execute Workflow Trigger → AI Agent (tools Consultar_pedidos_notion + Gmail) → Code in JavaScript (salida).

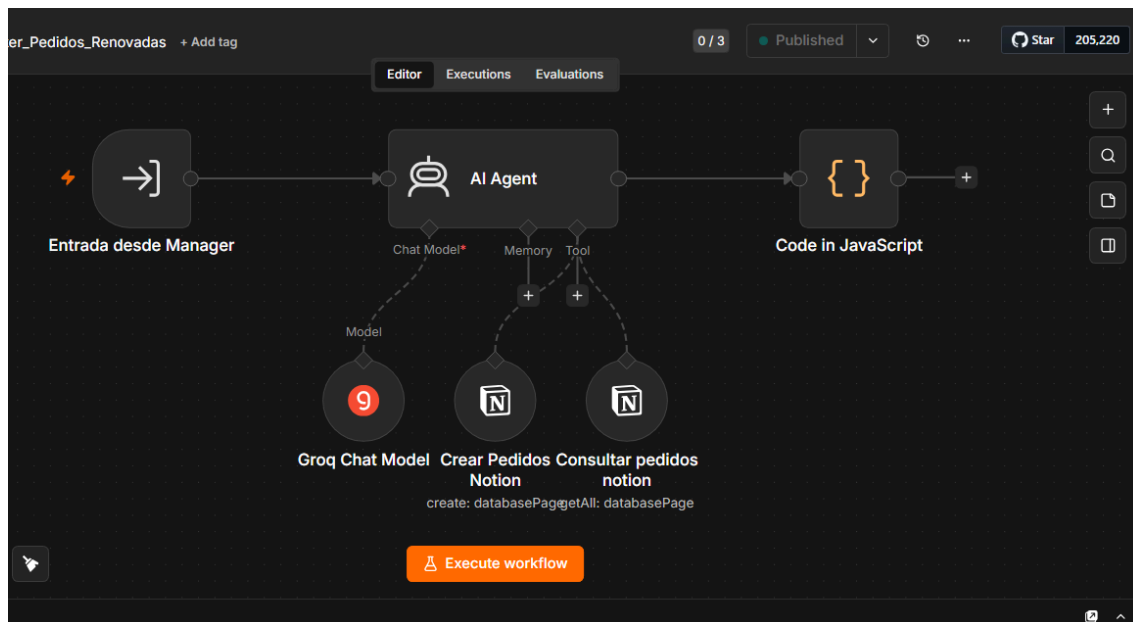


Figura 5 — Worker_Pedidos_Renovadas: Execute Workflow Trigger → AI Agent (tools Crear/Consultar Pedidos en Notion) → Code in JavaScript (salida).

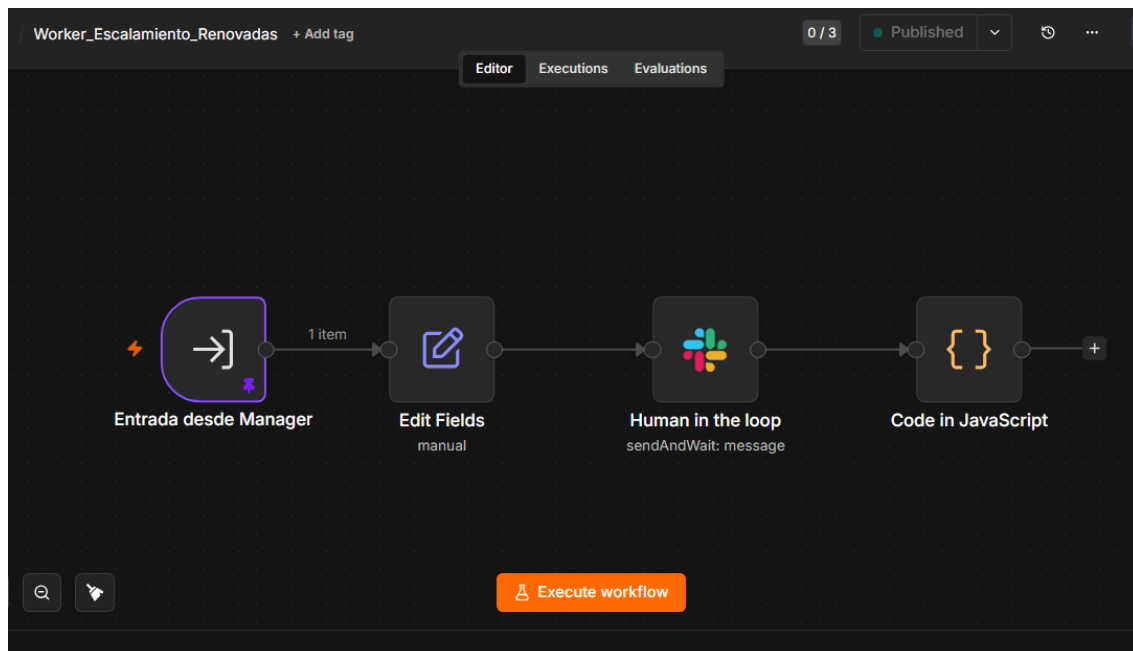


Figura 6 — Worker_Escalamiento_Renovadas: Execute Workflow Trigger → Edit Fields → Human in the loop (Slack, sendAndWait) → Code in JavaScript (salida).

4. Panel de ejecución — corrida exitosa de punta a punta

Prueba de regresión: se envió la consulta "¿tenés lentes marrones?" desde el chat del Manager. El Router clasificó {"intent":"catálogo","confidence":0.95,"risk":"LOW"}, el IF evaluó riesgo bajo (rama False), el Switch delegó a Worker_Catalogo, el Manager quedó en espera ("Wait For Sub-Workflow Completion") hasta recibir la respuesta del Worker, y el resultado volvió integrado al contexto y se mostró en el chat. Todo el recorrido se ejecutó en verde, sin errores de sintaxis.

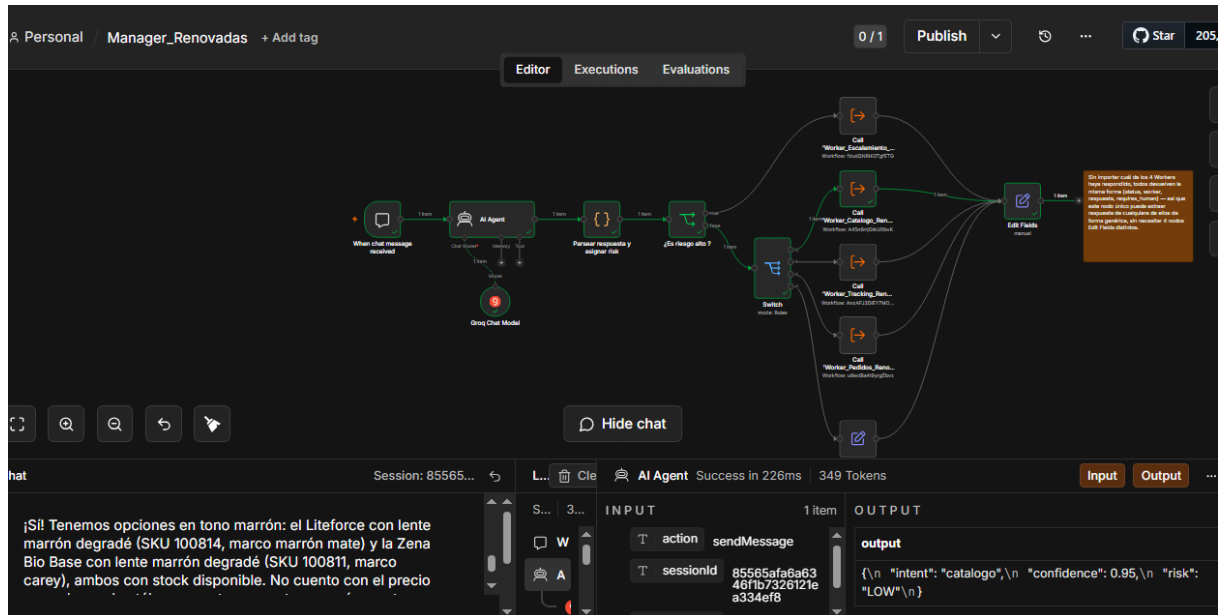


Figura 7 — Ejecución completa: respuesta final en el chat + panel de logs del Manager con el output del Router.

5. Esquema de datos (contrato de interfaz Manager ↔ Worker)

5.1 Manager → Worker (petición)

El Manager nunca reenvía el objeto crudo del Chat Trigger (que trae metadatos pesados: sessionId, action, headers, etc.). Antes de llamar a un Worker, un nodo Code parsea la respuesta del Router y arma un JSON mínimo de 4 campos — así se evita el antipatrón de "data stuffing":

```
{
  "original_input": "quiero el código de seguimiento de mi pedido",
  "intent": "tracking",
  "confidence": 0.85,
  "risk": "LOW"
}
```

5.2 Worker → Manager (respuesta)

Los 4 Workers, sin importar cuál haya respondido, devuelven exactamente la misma forma de objeto (contrato de salida homogéneo), lo que permite que el Manager tenga un único nodo Edit Fields al final que extrae "respuesta" sin necesitar lógica distinta por Worker:

```
{
  "status": "success",
  "worker": "TRACKING",
  "respuesta": "Hola Juan, tu pedido está pendiente. El Responsable de Envío aún no cargó el código...",
  "requires_human": false
}
```

El campo "status" puede tomar el valor "error" cuando el nodo Code del Worker no logra parsear un JSON válido en la respuesta del modelo — en ese caso arma él mismo un objeto de contingencia con "requires_human": true en vez de dejar caer la ejecución, cubriendo el escenario de "falta de manejo de fallos en el Worker" señalado como antipatrón en la consigna.

6. Criterio de enrutamiento

El Router (AI Agent sin tools, dentro del Manager) clasifica cada mensaje en una taxonomía cerrada de 5 valores de intent: catálogo, tracking, pedidos, escalamiento, saludo, junto con un confidence (0–1) y un risk (LOW / HIGH).

El enrutamiento ocurre en dos etapas secuenciales:

- 1) Nodo IF — filtro de riesgo: si `risk.trim().toUpperCase() == "HIGH"` (reclamos, cancelaciones, pagos, o cualquier caso sensible detectado por el Router), la rama True conecta directo a Worker_Escalamiento, sin pasar por el Switch — priorizando la seguridad del negocio sobre la intención literal del mensaje.
- 2) Nodo Switch — enrutamiento por intención: solo los casos de riesgo bajo llegan acá. Se evalúa intent contra 4 reglas exactas (catálogo / tracking / pedidos / saludo) y cada una dispara su Execute Workflow correspondiente. El caso "saludo" no llama a ningún Worker: responde con un mensaje de bienvenida fijo (identidad de marca "Rena"), evitando gastar tokens en una llamada innecesaria.

Como red de seguridad, si el nodo Code del Router no logra parsear el JSON del modelo, asigna por defecto risk: "HIGH", forzando que cualquier clasificación fallida termine en revisión humana en vez de caer en un Worker equivocado sin que nadie se entere.

7. Persistencia visual (observabilidad)

El log de trazabilidad se resuelve con nodos de Slack: cada Worker que interactúa con el cliente (Catálogo, Tracking, Pedidos) puede reportar su actividad a un canal de soporte, y Worker_Escalamiento usa un nodo "Human in the loop" (Slack Send and Wait) que registra el motivo del caso y pausa el flujo hasta que un humano apruebe o rechace la acción antes de que el Manager continúe.

